

[See Topic 09 → Heap Masterclass](#)

 **GOAL:** Master recursive tree thinking, DFS Call Stack timelines, BFS Queue level snapshots, BST properties, and solve all NeetCode Tree problems!

REUSABLE TREE & BST UTILITIES

TreeNode Definition & Construction:

```
public class TreeNode {
    public int val;
    public TreeNode left, right;
    public TreeNode(int val) { this.val = val; }
}
```

Height & Depth Helper Function:

```
int height(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(height(root.left), height(root.right));
}
```

AHA MOMENT #1: TREES ARE RECURSIVE BY DEFINITION

A Binary Tree is either 'null' OR a root node pointing to a ****Left Subtree**** and a ****Right Subtree****. Because every subtree is a valid tree itself, tree algorithms are naturally solved using ****Recursion (DFS)**** or ****Queue Level Snapshots (BFS)****!

1 CORE TREE TERMINOLOGY

- **Root:** Topmost node with zero parent.
- **Leaf:** Node with zero children ('left == null && right == null').
- **Height:** Max edges from node down to a leaf (Root height = H).
- **Depth:** Edges from root down to node (Root depth = 0).
- **Balanced:** Left & Right subtree heights differ by ≤ 1 ($O(\log N)$ height).

2 JAVA TREENODE NODE STRUCTURE

```
public class TreeNode {
    public int val;
    public TreeNode left;
    public TreeNode right;
    public TreeNode(int val) { this.val = val; }
}
```

TREE TYPES: BALANCED VS SKEWED

1. BALANCED TREE (Height $O(\log N)$): **2. SKEWED TREE (Height $O(N)$ - Linked List):** [10] [10] / \ \ [5] [15] [15] / \ / \ \ [2] [7] [12] [20] [20]

 **KEY TAKEAWAY:** In a balanced tree, operations run in $O(\log N)$ time. In a skewed tree, operations degrade to $O(N)$!

- 💡 AHA MOMENT #2: THE 3 DFS ORDERS (PRE, IN, POST)
- **Preorder (Root → Left → Right):** Use for cloning, serializing, and passing values down.
 - **Inorder (Left → Root → Right):** Yields ****sorted order**** in a Binary Search Tree (BST)!
 - **Postorder (Left → Right → Root):** Use for bottom-up calculation (compute children first, then return to parent)!

📊

DFS TRAVERSAL COMPARISON TABLE

Traversal	Order	Primary FAANG Use Case
Preorder	Root → Left → Right	Serialize/Deserialize, Clone Tree, Top-Down Path Sum
Inorder	Left → Root → Right	BST Validation, Kth Smallest in BST, Sorted Processing
Postorder	Left → Right → Root	Max Depth, Diameter, Tree Deletion, Bottom-Up Path Sum

🌀

RECURSIVE CALL STACK TIMELINE DEMO (TREE: 3 / 9, 20 / 15, 7)

DFS CALL STACK TIMELINE (INORDER): 1. `inorder(3)` → Calls `inorder(9)` 2. `inorder(9)` → Calls `inorder(null)` → returns → Visit [9] → Calls `inorder(null)` → returns 3. Back to `inorder(3)` → Visit [3] → Calls `inorder(20)` 4. `inorder(20)` → Calls `inorder(15)` → Visit [15] → returns → Visit [20] → Calls `inorder(7)` → Visit [7] Output Order: [9, 3, 15, 20, 7]

🚩 **INORDER BST RULE:** An Inorder traversal of a valid BST ALWAYS produces a strictly increasing sorted sequence!

 AHA MOMENT #3: THE LEVEL SNAPSHOT PATTERN

To traverse a tree level-by-level (BFS), use a ****FIFO Queue****. To separate nodes by level, freeze the queue size at the start of each level:

```
int size = q.size(); for (int i = 0; i < size; i++) { TreeNode curr = q.poll(); ... }
```

This guarantees that all nodes in the current loop belong strictly to the **SAME** level!

 VISUALIZING BFS QUEUE EVOLUTION (LEVEL-BY-LEVEL)

BFS QUEUE EVOLUTION: Level 0: Queue = [3] → Poll 3 → Offer children 9, 20 → Level 0 Result = [3] Level 1: Queue = [9, 20] → Poll 9, 20 → Offer children 15, 7 → Level 1 Result = [9, 20] Level 2: Queue = [15, 7] → Poll 15, 7 → Level 2 Result = [15, 7]

 STANDARD JAVA BFS LEVEL ORDER TEMPLATE

```
Queue<TreeNode> q = new ArrayDeque<>();
if (root != null) q.offer(root);
while (!q.isEmpty()) {
    int levelSize = q.size(); // Freeze current level size!
    for (int i = 0; i < levelSize; i++) {
        TreeNode curr = q.poll();
        if (curr.left != null) q.offer(curr.left);
        if (curr.right != null) q.offer(curr.right);
    }
}
```

 **KEY TAKEAWAY:** Use BFS whenever a problem asks for "shortest distance", "level-by-level", or "nearest leaf"!

💡 **AHA MOMENT #4: BST SEARCH DISCARDING & LCA ELEGANCE**

- **BST Property:** For every node, 'left.val < node.val < right.val'. This allows discarding half the tree at each step during search in $O(\log N)$ time!
- **BST LCA:** If 'p' and 'q' are both smaller than 'root' → LCA is in left subtree. If both larger → right subtree. Otherwise, 'root' is the split point (LCA)!

1 **BST SEARCH & INSERT ($O(\log N)$)**

```
public TreeNode searchBST(TreeNode root, int val) {
    if (root == null || root.val == val) return root;
    return val < root.val ?
        searchBST(root.left, val) : searchBST(root.right, val);
}
```

2 **BST LCA 1-LINE SPLIT LOGIC**

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p,
TreeNode q) {
    if (p.val < root.val && q.val < root.val) return
lowestCommonAncestor(root.left, p, q);
    if (p.val > root.val && q.val > root.val) return
lowestCommonAncestor(root.right, p, q);
    return root; // Split point is LCA!
}
```

🚩 **KEY TAKEAWAY:** In a general Binary Tree (non-BST), LCA requires bottom-up postorder bubbling checking if left and right returns are non-null!

 **TREE INTERVIEW DECISION TREE (MERMAID)**

 **PRINTABLE TREE & BST CHEAT SHEET**

Pattern Type	Target Problem	Primary Method	Complexity
Bottom-Up DFS	Max Depth (LC 104)	<code>1 + Math.max(left, right)</code>	Time: $O(N)$, Space: $O(H)$
Top-Down DFS	Good Nodes (LC 1448)	Pass <code>maxSoFar</code> in parameters	Time: $O(N)$, Space: $O(H)$
BFS Level Order	Level Order (LC 102)	Queue Snapshot <code>q.size()</code>	Time: $O(N)$, Space: $O(W)$
BST Validation	Validate BST (LC 98)	Range Check <code>[min, max]</code>	Time: $O(N)$, Space: $O(H)$
LCA in BST	LCA BST (LC 235)	Compare <code>p.val</code> and <code>q.val</code> with root	Time: $O(H)$, Space: $O(H)$

 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!

1. LEETCODE 226 — INVERT BINARY TREE

 **Easy** ★★★★★ **Postorder Swap** [Google](#) [Amazon](#)

Problem:

Invert (mirror) a binary tree so left and right children are swapped.

CALL STACK TIMELINE (TREE: 4 / 2, 7): 1. invert(4) → Calls invert(2) 2. invert(2) → Swaps 2's children → returns 2 3. invert(7) → Swaps 7's children → returns 7 4. Swaps 4's left and right subtrees → returns 4!

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode left = invertTree(root.left);
    TreeNode right = invertTree(root.right);
    root.left = right;
    root.right = left;
    return root;
}
```

2. LEETCODE 104 — MAXIMUM DEPTH OF BINARY TREE

 **Easy** ★★★★★ **Bottom-Up DFS** [Amazon](#) [Meta](#)

Problem:

Return maximum depth (number of nodes along longest path from root to leaf).

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

3. LEETCODE 100 — SAME TREE

 **Easy** ★★★★★ **Dual Tree DFS** [Amazon](#)

Problem:

Check if two binary trees 'p' and 'q' are structurally identical with same values.

```
public boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val) return false;
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

 **AHA MOMENT (LC 104):** Bottom-up DFS computes children depths first, then returns '1 + max(left, right)' to parent!

4. LEETCODE 572 — SUBTREE OF ANOTHER TREE

Easy
★★★★★
Subtree Matching
Amazon

Problem:
 Return `true` if tree `subRoot` is a valid subtree of `root`.

```

public boolean isSubtree(TreeNode root, TreeNode subRoot) {
    if (root == null) return false;
    if (isSame(root, subRoot)) return true;
    return isSubtree(root.left, subRoot) || isSubtree(root.right,
subRoot);
}
private boolean isSame(TreeNode s, TreeNode t) {
    if (s == null && t == null) return true;
    if (s == null || t == null || s.val != t.val) return false;
    return isSame(s.left, t.left) && isSame(s.right, t.right);
}

```

5. LEETCODE 110 — BALANCED BINARY TREE

Easy
★★★★★
Bottom-Up Height Checking
Amazon
Meta

Problem:
 Determine if binary tree is height-balanced (left/right depth diff ≤ 1).

```

public boolean isBalanced(TreeNode root) {
    return checkHeight(root) != -1;
}
private int checkHeight(TreeNode node) {
    if (node == null) return 0;
    int left = checkHeight(node.left);
    if (left == -1) return -1;
    int right = checkHeight(node.right);
    if (right == -1) return -1;
    if (Math.abs(left - right) > 1) return -1; // Unbalanced sentinel!
    return 1 + Math.max(left, right);
}

```

💡 **AHA MOMENT (LC 110):** Returning `-1` as a sentinel value prunes early and avoids recalculating height at every node ($O(N)$ vs $O(N^2)$)!



6. LEETCODE 102 — BINARY TREE LEVEL ORDER TRAVERSAL

Medium ★★★★★ BFS Queue Level Snapshot Amazon Meta

Problem:

Return level order traversal of nodes' values (left to right, level by level).

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int levelSize = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            TreeNode curr = q.poll();
            level.add(curr.val);
            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
        res.add(level);
    }
    return res;
}
```

7. LEETCODE 199 — BINARY TREE RIGHT SIDE VIEW

Medium ★★★★★ BFS Last Element / Preorder DFS Meta Amazon

Problem:

Return values of nodes visible when standing on right side of tree.

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int levelSize = q.size();
        for (int i = 0; i < levelSize; i++) {
            TreeNode curr = q.poll();
            if (i == levelSize - 1) res.add(curr.val); // Last node in
level!
            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
    }
    return res;
}
```

💡 **AHA MOMENT (LC 199):** In BFS Level Order, the LAST element polled in each level ('i == levelSize - 1') is the rightmost visible node!



8. LEETCODE 1448 — COUNT GOOD NODES IN BINARY TREE

Medium ★★★★★ Top-Down DFS Parameter Microsoft

Problem:

Node X is "good" if path from root to X contains no nodes with value $> X$.

```
public int goodNodes(TreeNode root) {
    return dfs(root, root.val);
}

private int dfs(TreeNode node, int maxSoFar) {
    if (node == null) return 0;
    int count = (node.val >= maxSoFar) ? 1 : 0;
    maxSoFar = Math.max(maxSoFar, node.val);
    count += dfs(node.left, maxSoFar);
    count += dfs(node.right, maxSoFar);
    return count;
}
```

9. LEETCODE 98 — VALIDATE BINARY SEARCH TREE

Medium ★★★★★ Range Boundary Check Amazon Meta

Problem:

Validate if binary tree satisfies strict BST property $\text{'left' < root < right'}$.

```
public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private boolean validate(TreeNode node, Integer min, Integer max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max != null && node.val >= max)) {
        return false; // Out of bounds!
    }
    return validate(node.left, min, node.val) && validate(node.right, node.val, max);
}
```

**AHA MOMENT (LC 98):** Checking only immediate children is wrong! Pass valid range '[min, max]' boundaries down through recursion!

10. LEETCODE 230 — KTH SMALLEST ELEMENT IN A BST

 **Medium** ★★★★★ **Inorder Monotonic Traversal** [Amazon](#) [Meta](#)

Problem:

Find K^{th} smallest value in a BST using Inorder traversal.

```
private int count = 0, result = -1;
public int kthSmallest(TreeNode root, int k) {
    inorder(root, k); return result;
}
private void inorder(TreeNode node, int k) {
    if (node == null || count ≥ k) return;
    inorder(node.left, k);
    count++;
    if (count == k) { result = node.val; return; }
    inorder(node.right, k);
}
```

11. LEETCODE 236 — LOWEST COMMON ANCESTOR OF BINARY TREE

 **Medium** ★★★★★ **Bottom-Up Postorder Bubbling** [Amazon](#) [Meta](#)

Problem:

Find LCA of nodes 'p' and 'q' in a general binary tree.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left ≠ null && right ≠ null) return root; // Root is split point!
    return left ≠ null ? left : right;
}
```

 **MEMORY BOX:** If both 'left' and 'right' subtrees return non-null, current 'root' is the Lowest Common Ancestor!

12. LEETCODE 105 — CONSTRUCT TREE FROM PREORDER & INORDER

🔥 Medium ★★★★★ Preorder Root + Inorder Split [Amazon](#)

Problem:

Build binary tree from preorder and inorder traversal arrays.

```
private int preIdx = 0;
private Map<Integer, Integer> inMap = new HashMap<>();
public TreeNode buildTree(int[] preorder, int[] inorder) {
    for (int i = 0; i < inorder.length; i++) inMap.put(inorder[i], i);
    return helper(preorder, 0, inorder.length - 1);
}
private TreeNode helper(int[] pre, int inStart, int inEnd) {
    if (inStart > inEnd) return null;
    int rootVal = pre[preIdx++];
    TreeNode root = new TreeNode(rootVal);
    int inIndex = inMap.get(rootVal);
    root.left = helper(pre, inStart, inIndex - 1);
    root.right = helper(pre, inIndex + 1, inEnd);
    return root;
}
```

🔥 Medium ★★★★★ Reverse Postorder / Morris [Meta](#)

Problem:

Flatten tree into a single right-skewed linked list in-place.

```
private TreeNode prev = null;
public void flatten(TreeNode root) {
    if (root == null) return;
    flatten(root.right); // Right first
    flatten(root.left); // Left second
    root.right = prev;
    root.left = null;
    prev = root;
}
```

💡 **AHA MOMENT (LC 114):** Traversing Right → Left → Root (Reverse Postorder) makes flattening a simple pointer assignment 'root.right = prev'!

14. LEETCODE 124 — BINARY TREE MAXIMUM PATH SUM
 **Hard ★★★★★** Bottom-Up Postorder + Max Update [Meta](#) [Amazon](#) [Google](#)
Problem:

Max path sum between any two nodes in binary tree.

```
private int maxSum = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root) {
    gain(root); return maxSum;
}
private int gain(TreeNode node) {
    if (node == null) return 0;
    int leftGain = Math.max(0, gain(node.left));
    int rightGain = Math.max(0, gain(node.right));
    maxSum = Math.max(maxSum, node.val + leftGain + rightGain); // Split
    path
        return node.val + Math.max(leftGain, rightGain); //
    Single branch
}
```

15. LEETCODE 297 — SERIALIZE AND DESERIALIZE BINARY TREE
 **Hard ★★★★★** Preorder DFS String Queue [Amazon](#) [Google](#)
Problem:

Convert tree to string and back to identical tree structure.

```
public String serialize(TreeNode root) {
    if (root == null) return "N,";
    return root.val + "," + serialize(root.left) +
        serialize(root.right);
}
public TreeNode deserialize(String data) {
    Queue<String> q = new ArrayDeque<>(Arrays.asList(data.split(",")));
    return build(q);
}
private TreeNode build(Queue<String> q) {
    String val = q.poll();
    if (val.equals("N")) return null;
    TreeNode node = new TreeNode(Integer.parseInt(val));
    node.left = build(q); node.right = build(q);
    return node;
}
```

 **FAANG COACH TIP:** For Max Path Sum, return ONLY a single branch `node.val + max(left, right)` up to parent, while updating global `maxSum` with the split path `node.val + left + right`!

16. LEETCODE 314 / 987 — VERTICAL ORDER TRAVERSAL

Hard ★★★★★ BFS + Column Coordinate Map Meta Amazon

★ **FLOWCHART:** Assign root column = 0. Left child = col-1, Right child = col+1. Group by column!

Problem:

Return vertical order traversal of binary tree from leftmost column to rightmost column.

```
public List<List<Integer>> verticalOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Map<Integer, List<Integer>> map = new TreeMap<>(); // Keep columns
    sorted!
    Queue<TreeNode> nodeQ = new ArrayDeque<>();
    Queue<Integer> colQ = new ArrayDeque<>();
    nodeQ.offer(root); colQ.offer(0);
    while (!nodeQ.isEmpty()) {
        TreeNode curr = nodeQ.poll();
        int col = colQ.poll();
        map.computeIfAbsent(col, k → new ArrayList<>()).add(curr.val);
        if (curr.left != null) { nodeQ.offer(curr.left); colQ.offer(col
- 1); }
        if (curr.right != null) { nodeQ.offer(curr.right);
colQ.offer(col + 1); }
    }
    return new ArrayList<>(map.values());
}
```

★ **KEY TAKEAWAY:** Pairing node Queue with Column Queue (`colQ`) keeps coordinates perfectly synchronized during BFS!

★ THE 4 GOLDEN LAWS OF TREES & BSTS

1. **Subtrees are Trees:** Always write base cases for `root == null` first!
2. **Inorder BST Sorted Property:** Inorder traversal of valid BST is ALWAYS monotonically ascending.
3. **BFS Level Snapshot:** Freeze size `int size = q.size()` before processing levels.
4. **Bottom-Up vs Top-Down DFS:** Use Bottom-Up (postorder) when returning subtree computations up to parent!

🚫 WHEN RECURSION FAILS (DO NOT USE!)

- **Deep Skewed Trees (\$N > 10^4\$):** Unchecked deep recursion causes `StackOverflowError` → Use ****Iterative Stack/Queue****!

📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

 Easy LC 226 Invert Tree	 Easy LC 104 Max Depth	 Easy LC 100 Same Tree
 Easy LC 572 Subtree	 Easy LC 110 Balanced Tree	 Med LC 102 Level Order
 Med LC 199 Right Side View	 Med LC 1448 Good Nodes	 Med LC 98 Validate BST
 Med LC 230 Kth Smallest	 Med LC 235/236 LCA	 Med LC 105 Build Tree
 Hard LC 124 Max Path Sum	 Hard LC 297 Serialize/Deserialize	 Hard LC 314 Vertical Order

👉 **Next Master Topic: Topic 11** → **Tries Masterclass**

🇬🇧 **TOPIC 10 TREES MASTERCLASS COMPLETE!** Turn the page to **Topic 11: Tries Masterclass!**